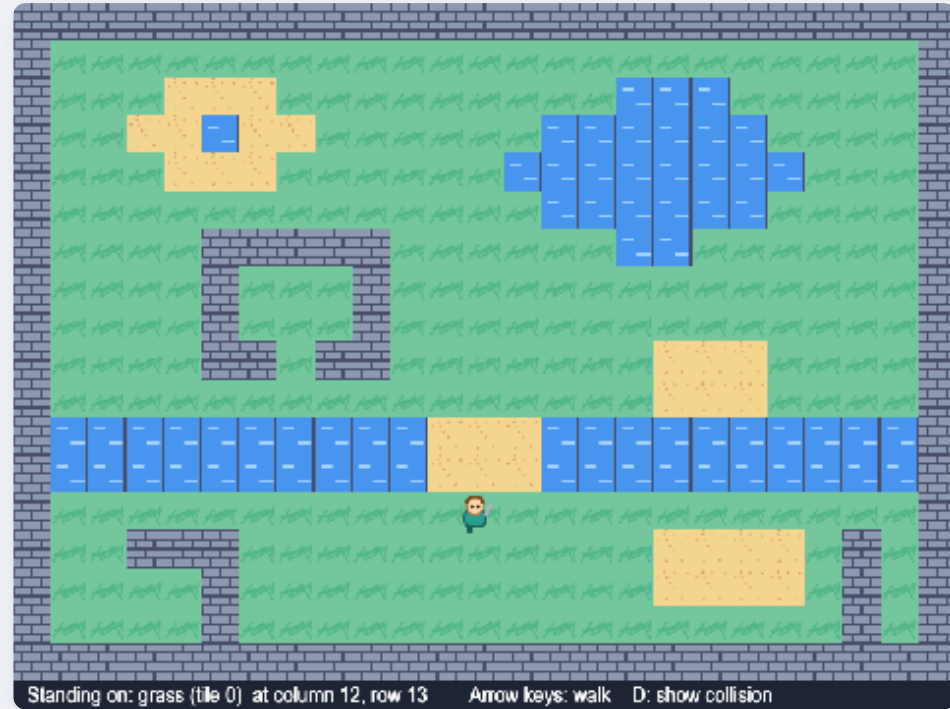


Chapter 10

Tilemaps

Worlds built from tiles - in code



Today

- why games use **tiles**; tilesets and tile **indexes**
- a map from a **2D array**: map, tileset, layer
- **collision** by tile index
- finding tiles; **pixels and tiles**
- changing the map while the game runs
- a world **bigger than the screen**: camera follow, bounds, a minimap
- free movement or **grid** movement?

Why tiles?

- **memory:** one small picture, reused. A 60 x 40 level as one picture: ~10 MB
- **level design:** a level is a grid of numbers - change a number, change the level
- **rules:** "is this a wall?" is a question about a number, not about pixels

Tilesets in the library (all 32 x 32): `simple_tiles.png` (4), `dungeon_tiles.png` (16), `platform_tiles.png` (16)

Tiles, indexes, and the level

A tileset is one picture; a map is a grid of numbers that pick from it

simple_tiles.png - 128 x 32 pixels, four 32 x 32 tiles



0
grass



1
water



2
wall



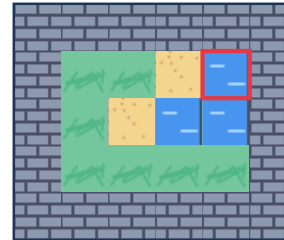
3
sand

each tile's INDEX is its place in the picture, counting from 0 - left to right, then row by row in a taller tileset

-1 means "no tile": nothing is drawn

```
const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2],
  [2, 0, 0, 3, 1, 2],
  [2, 0, 3, 1, 1, 2],
  [2, 0, 0, 0, 0, 2],
  [2, 2, 2, 2, 2, 2],
];
```

make.tilemap

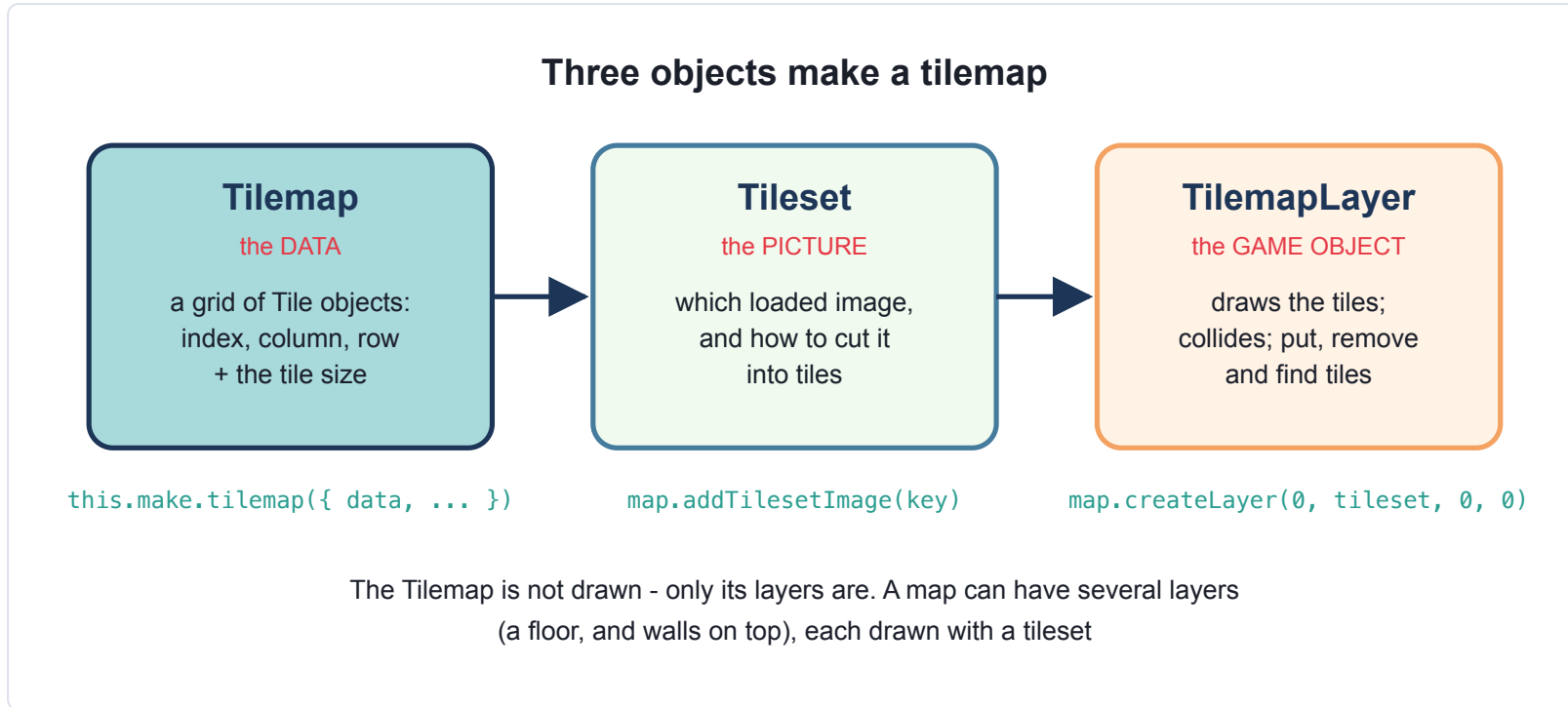


LEVEL[1][4] is 1:
row 1, column 4
is water

rows first: LEVEL[row][column] - the array looks like the map

LEVEL[row][column] - rows first. -1 = no tile

Three objects make a tilemap



Five steps

```
const map = this.make.tilemap({ data: LEVEL, tileWidth: TILE_SIZE, tileHeight: TILE_SIZE });

const tileset = map.addTilesetImage(TILES_KEY)!;

this.layer = map.createLayer(0, tileset, 0, 0) as Phaser.Tilemaps.TilemapLayer;

this.layer.setCollision([WATER, WALL]);

this.physics.add.collider(this.player, this.layer);
```

- `!` - "not `null` this time"
- `as` - `createLayer` could also make a GPU layer; we know it did not

Collision

```
// a body smaller than the picture, centred on it  
this.setSize(BODY_SIZE, BODY_SIZE);
```

- a 32 x 32 body through a 32-pixel door: pixel-perfect or stuck
- `setCollision` is **by index**, and remembered - tiles put later collide too
- press **D**: `layer.renderDebug(graphics, {...})`



Try it: Array Map - which tile am I on?

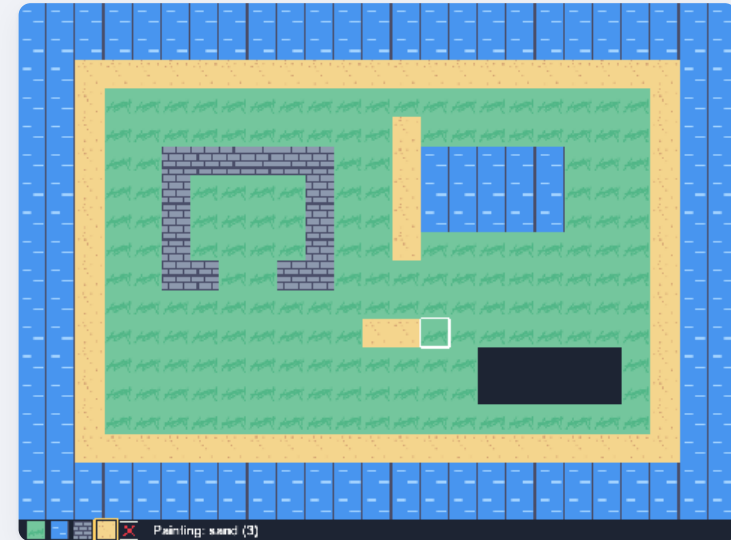
```
const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);
if (tile) {
  const name = TILE_NAMES[tile.index];
  ...
}
```

- `tile.x`, `tile.y` - column and row, **in tiles**; `pixelX`, `pixelY` - pixels
- walk round `ch10_array_map`; watch the strip; press D
- put a wall in the doorway (one number in `level.ts`)

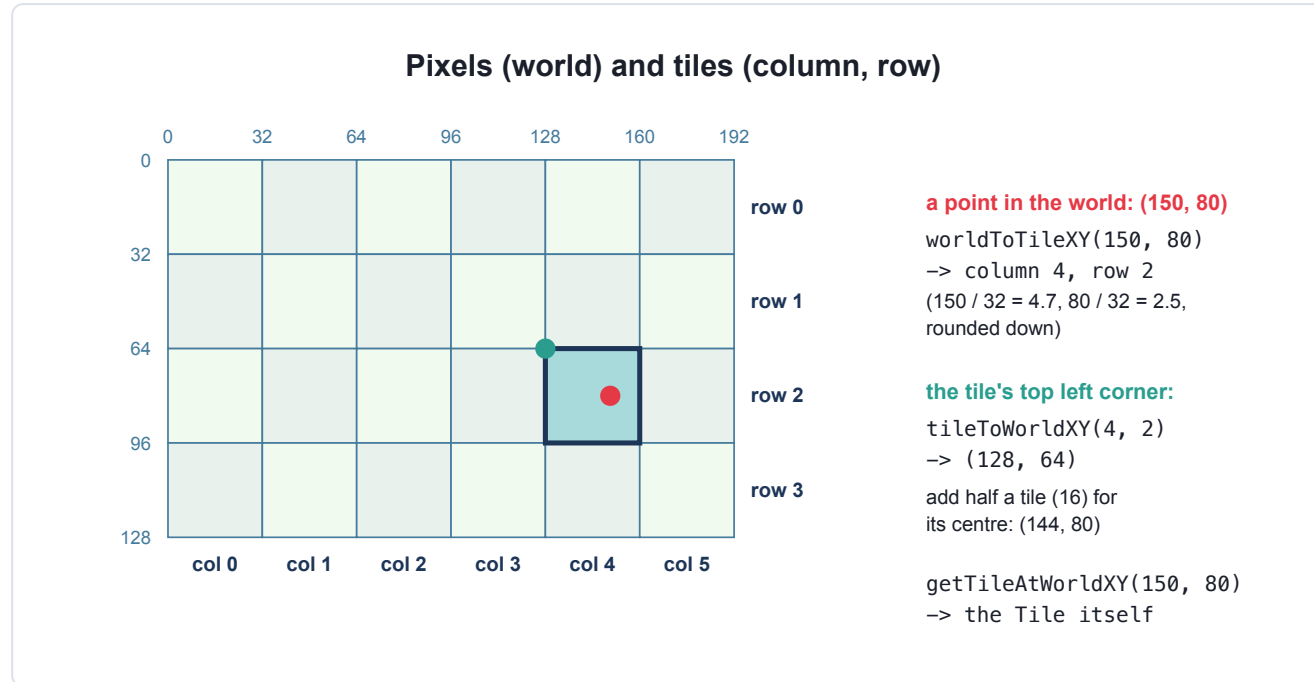
The tile painter: put and remove

```
this.layer = map.createBlankLayer("ground", tileset!);  
this.layer.fill(GRASS);  
...  
if (this.current === EMPTY) {  
    this.layer.removeTileAt(cell.x, cell.y);           // leaves a hole: noth  
} else {  
    this.layer.putTileAt(this.current, cell.x, cell.y);  
}
```

- `EMPTY` is `-1` : no tile. `0` is grass!



Pixels and tiles



`pointer.worldX`, not `pointer.x` - it allows for the camera

Save, load, print

```
localStorage.setItem(SAVE_KEY, JSON.stringify(this.toArray()));  
...  
let data: unknown = null;  
...  
if (!this.isMap(data)) {  
  ...  
}  
this.layer.putTilesAt(data, 0, 0);  
...  
private isMap(value: unknown): value is number[][] {
```

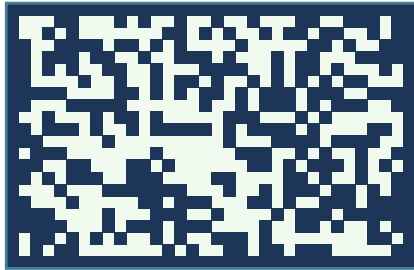
- `unknown` - could be anything: check it
- `value is number[][]` - a **type guard**
- P prints the map as TypeScript - paste it into `ch10_array_map`

Try it: Tile Painter

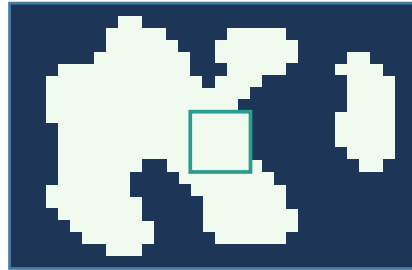
- paint an island: 1-4 choose, 0 erases, drag to paint
- S, C, L - save, clear, load
- P, then look in the browser's console (F12)
- paste the result into `ch10_array_map/src/level.ts` and walk round it

A cave made in code

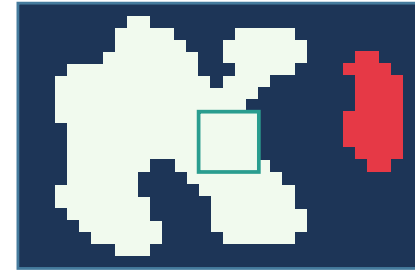
Making a cave: noise, smoothed, then cut-off caves filled in



1. random rock (47%)



2. smoothed 5 times



3. unreachable floor filled

dark = rock, light = floor, green = the cleared start, red = floor the player could never reach, now rock

- `Cave` is plain data - no Phaser. **Model**, not view
- the scene decides what it looks like

Two layers

```
const floor = map.createBlankLayer("floor", tileset!);
floor.weightedRandomize([
  { index: FLOOR, weight: 12 },
  { index: FLOOR_CRACKED, weight: 1 },
  { index: FLOOR_MOSSY, weight: 1 },
]);

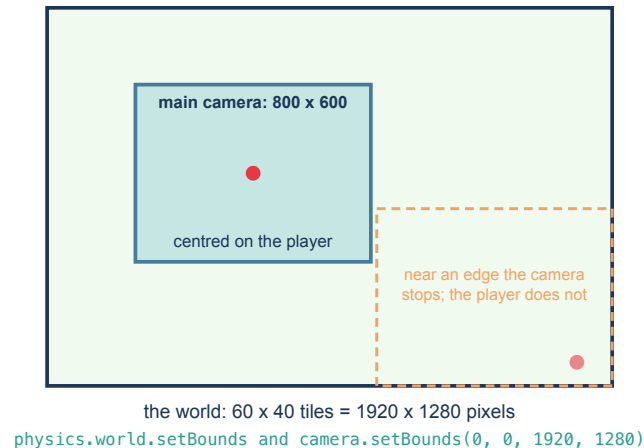
const walls = map.createBlankLayer("walls", tileset!);
walls.putTilesAt(this.wallTiles(cave), 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not empty collides
```

Walls: -1 where the cave is open, so the floor shows through

The camera

```
this.physics.world.setBounds(0, 0, width, height);  
this.cameras.main.setBounds(0, 0, width, height);  
this.cameras.main.startFollow(this.player, true);    // true: round to whole pixels - no shimmer  
this.hudText.setScrollFactor(0);                    // stays put on the screen while the camera moves
```

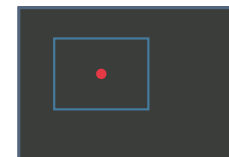
A world bigger than the screen



startFollow(player, true)
the camera's scrollX / scrollY change
every frame to keep the player centred

setBounds(...)
never shows anything outside the map

hud.setScrollFactor(0)
stays in the same place on screen



a second camera,
zoom 0.1: the
whole world in
192 x 128 pixels

cameras.add(x, y, 192, 128)

A minimap is just another camera

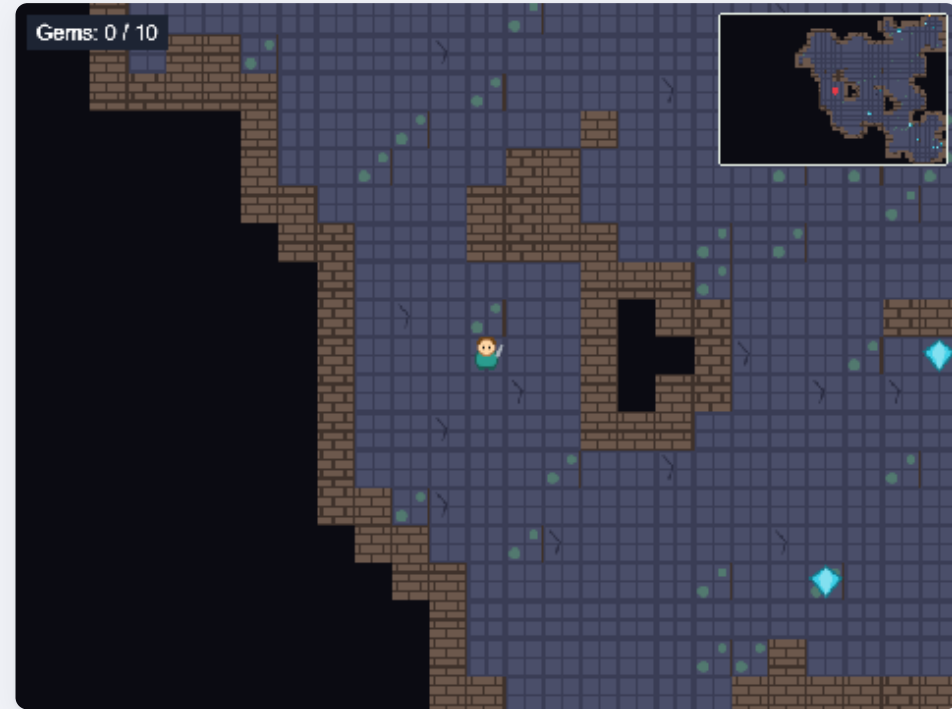
```
const minimap = this.cameras.add(MINI_X, MINI_Y, MINI_WIDTH, MINI_HEIGHT);
minimap.setZoom(MINI_ZOOM);
minimap.centerOn(width / 2, height / 2);

this.marker = this.add.circle(this.player.x, this.player.y, MARKER_RADIUS, 0xe63946);
this.cameras.main.ignore(this.marker);
minimap.ignore([frame, this.hudText]);
```

Every camera draws everything - unless told to `ignore` it

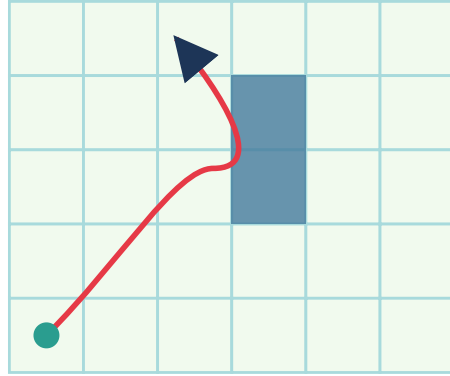
Try it: Scrolling World

- find the ten gems; R for a new cave
- the same `Player` class as `ch10_array_map` - unchanged
- what happens without `minimap.ignore(...)` ? Without `setScrollFactor(0)` ?

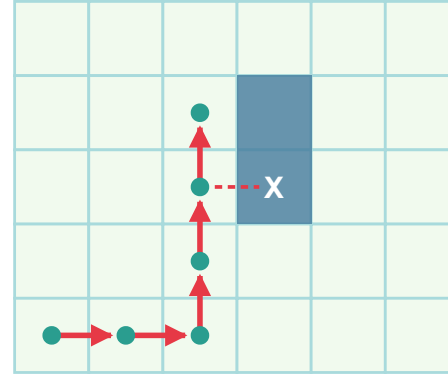


Free or grid movement?

Free movement and grid movement



FREE: velocity from the keys;
any pixel; the collider stops you
(this chapter's projects)



GRID: one whole tile per move, centre
to centre; look at the next tile first
(a wall or water there: do not move)

Zelda? Sokoban? Pac-Man? A roguelike?

Summary

- `tileset` = one picture; index from 0; `-1` = empty; `data[row][column]`
- `make.tilemap` (`data`) -> `addTilesetImage` (`picture`) -> `createLayer` (`game object`)
- `setCollision` + `physics.add.collider`
- `getTileAt(WorldXY)`, `worldToTileXY`, `tileToWorldXY` (top left corner)
- `putTileAt`, `removeTileAt`, `putTilesAt`, `fill`, `weightedRandomize`
- `camera.setBounds`, `startFollow`, `setScrollFactor(0)`, `cameras.add`, `ignore`

Challenges

1. **A new island** - moat, second bridge, maze, new start
2. **Sand slows you down** - half speed on sand
3. **Eyedropper** - right-click picks up a tile
4. **Treasure tiles** - chests in the map, collected by walking into them
5. **A map from strings** - `#`, `~`, `.`, `s`, `@`
6. **Grid movement** - one tile at a time, with tweens

Next: **Chapter 11 - Tilemaps with Tiled**